

Python Control Flow: Logic, Loops & Syntax

A Comprehensive Student Guide from Basics to Advanced Concepts

Core Principle

Control flow dictates the execution sequence of statements in a Python program. Unlike languages that rely on curly braces {}, Python enforces block structure using exact **indentation** (typically 4 spaces). This guide details decision-making structures, repetition loops, and practical task solutions.

1. Syntax & Boolean Operators

Decision making relies on evaluating expressions to True or False. The table below outlines the core operators used in conditional control flow:

Operator	Type	Description	Example
==	Comparison	Equal to	x == 5
!=	Comparison	Not equal to	x != 0
< , >	Comparison	Less than / Greater than	age >= 18
and	Logical	Returns True if BOTH conditions are True	(x > 0) and (y > 0)
or	Logical	Returns True if AT LEAST ONE condition is True	(x < 0) or (y < 0)
not	Logical	Inverts the boolean value	not (x == y)

2. Conditional Statements (if, elif, else)

Conditional statements evaluate boolean conditions from top to bottom. As soon as one condition evaluates to True, its indented block executes and the remaining clauses are skipped.

```
# Example: Evaluating letter grades using if-elif-else
score = 85

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B") # Executes! (85 >= 80 is True)
elif score >= 70:
    print("Grade: C")
else:
    print("Grade: F")
```

3. Definite Iteration (for Loops & range)

A for loop iterates over a defined sequence (list, string, tuple) or a generated numerical sequence using range(start, stop, step).

```
# Iterating over a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# Using range(start, stop, step) -> generates 2, 4, 6, 8, 10
for i in range(2, 11, 2):
    print(f"Even number: {i}")
```

4. Indefinite Iteration (while Loops)

A while loop executes repeatedly as long as its governing condition remains True. **Important:** Always ensure a variable state update inside the loop body to avoid infinite execution.

```
# Countdown timer with explicit state update
count = 5

while count > 0:
    print(f"Countdown: {count}")
    count -= 1 # Crucial state update!

print("Blastoff!")
```

5. Advanced Control Flow & Special Clauses

Python provides special control statements and loop features for granular execution management:

- **break:** Immediately exits the loop, bypassing any remaining iterations.
- **continue:** Skips the rest of the current iteration and jumps directly to the next loop evaluation.
- **Loop else clause:** Executes *only* if the loop completes all iterations normally without hitting a break statement.

```
# Searching for a prime number using loop-else
target = 7

for i in range(2, target):
    if target % i == 0:
        print(f"{target} is not prime")
        break
else:
    print(f"{target} is a Prime Number!") # Executes because break was never triggered
```

6. Student Practice Tasks & Solutions

Task 1: Temperature Alert (if-elif-else)

Problem: Write a program where temperature > 30 prints 'Hot', between 15 and 30 prints 'Warm', and < 15 prints 'Cold'.

```
temp = 22
if temp > 30:
    print('Hot')
elif temp >= 15:
    print('Warm')
else:
    print('Cold')
```

Task 2: Sum of Even Numbers (for loop)

Problem: Calculate the sum of all even numbers from 1 to 20 using range().

```
total = 0
for num in range(2, 21, 2):
    total += num
print('Sum:', total) # Output: 110
```

Task 3: Password Validator (while loop)

Problem: Prompt the user until they enter the correct password ('python123').

```
password = ''
while password != 'python123':
    password = input('Enter password: ')
print('Access Granted!')
```

Task 4: Multiplication Table Grid (Nested Loops)

Problem: Print a 5x5 grid displaying multiplication tables for numbers 1 to 5.

```
for row in range(1, 6):
    for col in range(1, 6):
        print(f'{row * col:3}', end=' ')
    print()
```